

jordanvalnet/code-dev-intel.ts

code-dev-intel : un gain mesuré de 13 à 34 % sur la facture d'IA, zéro utilisateur et un seul contributeur — un actif technique, pas encore une entreprise

Self-hosted AI code intelligence for TypeScript — symbol resolution, structural search, duplicate detection, and dependency graphs via MCP server.

0 ÉTOILES GITHUB	0 FORKS	1 BUS FACTOR sur 12 semaines	AGPL-3.0 LICENCE
--	-------------------------------	---	--

Quel problème et quel marché ?

POINT CLÉ

Un problème réel et chiffrable — le coût des jetons d'un agent qui relit tout — attaqué par un produit local, sans aucun élément commercial adossé à ce jour.

Le problème est un problème de coût : un assistant IA qui travaille sur du code relit des fichiers entiers à chaque question, et chaque relecture se facture en jetons.

Le produit installe chez le client un service qui répond directement aux questions de structure — où est défini ce symbole, qui l'appelle, quels fichiers cassent si on le change — sans relecture complète du dépôt.

La cible annoncée : les équipes qui font travailler des agents IA sur une base TypeScript moyenne ou grande, et qui veulent que leur code ne sorte pas de chez elles.

La proposition de valeur est donc double, et les deux moitiés sont vendables séparément : moins de jetons consommés, et un index qui reste local plutôt que confié à un service d'indexation distant.

Ce que le relevé ne dit pas : la taille de ce marché, le prix pratiqué, le moindre client. Le dépôt ne contient aucun élément commercial — ni tarif, ni page produit, ni collecte de fonds déclarée.

Quelle traction ?

POINT CLÉ

Aucun signal de marché — 0 étoile, 0 copie, 1 version publiée en mars — face à une démonstration technique convaincante mais faite par l'auteur seul.

2026-03-26
v0.1.6

Nulle, au sens où un investisseur l'entend : 0 étoile et 0 copie du projet, presque six mois après sa création le 20 mars 2026.

Une seule version a été mise à disposition du public, la v0.1.6 du 26 mars 2026, six jours après l'ouverture du dépôt.

Or le journal interne du projet est à la version 0.5.0, datée du 4 septembre 2026, plus un lot non publié : cinq mois de travail que personne, dehors, ne peut installer.

Côté demande, une seule sollicitation extérieure est ouverte, sans un seul commentaire, et aucune n'a été traitée dans les trente derniers jours.

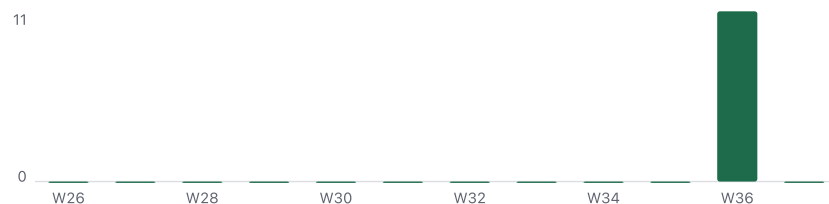
La seule preuve d'usage disponible est un banc d'essai conduit par l'auteur : sur une base de code de production, 11 agents sur 11 ont choisi l'outil spontanément, pour 13 à 34 % de jetons en moins sur les tâches difficiles, 27 % en moyenne¹.

Une mesure interne n'est pas une traction : elle dit que le produit marche, pas que quelqu'un le veut.

Quelle dynamique ?

POINT CLÉ

Onze semaines vides puis onze changements en une semaine : le rythme d'un projet personnel, pas celui d'un produit sous engagement.



Le projet avance par sursauts : 12 changements relevés sur les douze dernières semaines, dont 11 concentrés dans la seule semaine du 1er septembre, les onze semaines précédentes étant vides.

Le dernier changement date du 6 septembre 2026, soit quatre jours avant ce relevé : le projet est actif, il n'est pas régulier.

Ce qui est sorti lors de ce sursaut est un travail de mise en qualité, pas une nouveauté fonctionnelle : la chaîne de vérification tourne désormais sur les trois systèmes du marché.

La file est vide dans les deux sens : aucune proposition de changement en attente, aucun thème récurrent dans les demandes, une seule demande ouverte au total.

Donnée non relevée : les jalons de planification, à vérifier sur la page des jalons du projet².

Lecture d'investisseur : un rythme de projet personnel mené en soirées et en week-ends, cohérent avec un contributeur unique, incompatible avec un engagement de service.

Quelle équipe et quelle gouvernance ?

POINT CLÉ

Un contributeur, un compte personnel, aucun responsable ni financement déclaré : l'actif ne survit pas au départ de son auteur.

jordanvalnet

12

Une personne. Le compte jordanvalnet signe les 12 changements relevés sur douze semaines, et le nombre de personnes qui portent la moitié du travail est de 1.

Le dépôt vit sur un compte personnel, pas sur celui d'une société ni d'une fondation : il n'y a donc, en l'état, aucune structure à laquelle contracter.

La gouvernance est vide sur les trois marqueurs habituels : aucun responsable désigné pour le code, aucune politique de sécurité publiée, aucun financement déclaré.

Aucun contributeur extérieur n'est apparu depuis l'ouverture, alors que la procédure de contribution est écrite et publique.

Conséquence directe sur la valorisation : l'actif n'est pas séparable de son auteur. Ce qui s'achète ici est un savoir-faire attaché à une personne, pas une équipe qui continuerait sans elle.

Quelle concurrence ?

POINT CLÉ

La concurrence à battre est la recherche texte gratuite, et le produit la bat sur les jetons ; la barrière n'est pas l'idée, c'est l'exécution.

Donnée non relevée : la liste des projets comparables. Le dépôt ne déclare aucun mot-clé public, ce qui rend l'établissement d'une liste de comparables impossible sans recherche manuelle ; à faire sur la recherche publique de GitHub³.

Ce que le projet revendique face au marché, c'est le positionnement local : une alternative auto-hébergée aux services d'indexation de code distants, argument de conformité autant que de coût.

Le vrai concurrent, cependant, n'est pas un produit, c'est le comportement par défaut : la recherche texte et la lecture de fichiers, gratuites et déjà présentes dans tous les assistants.

C'est contre elle que le projet se chiffre, et le chiffre est le meilleur argument du dossier : une sortie environ 72 % plus compacte que son propre format précédent et environ 60 % plus compacte que la recherche texte de référence, sur un symbole à 43 usages.

Défendabilité : faible sur le principe — l'idée est reproductible par tout éditeur d'outillage IA — et réelle sur l'exécution, la résolution de modules par le compilateur TypeScript et la réutilisation du graphe entre deux sessions représentant plusieurs mois de travail difficile.

Licence, financement, risques ?

POINT CLÉ

Deux risques élevés, licence et homme-clé, tous deux réglables par négociation avec une seule personne ; la partie technique, elle, est propre.

- **license** AGPL-3.0 : copyleft, contraignant pour un usage commercial.
- **security** Pas de SECURITY.md : aucun canal déclaré pour signaler une faille.
- **bus_factor** Un seul contributeur porte la moitié des commits sur 12 semaines.
- **ci** Quatre workflows dans .github/workflows ; ci.yml enchaîne lint, type-check et tests sur ubuntu, windows et macos (node 20, 22, 24), puis release-smoke, indexer-smoke et scan sécurité, à cha...
- **deps** 5 dépendances d'exécution et 9 de développement dans package.json, verrou pnpm-lock.yaml, versions vulnérables épinglées via pnpm.overrides ; engines annonce node >=18 mais ci.yml précise q...

La licence est le point dur du dossier : AGPL-3.0, copyleft, qui oblige à publier son propre code dès lors que le logiciel est intégré à un produit vendu ou simplement exposé en réseau.

En usage interne, elle n'a aucune conséquence. Dans une thèse d'investissement qui suppose un produit commercialisé, elle en a une : soit l'auteur, seul détenteur des droits, relicencie, soit la thèse tombe.

Aucun financement n'est déclaré et aucune politique de sécurité n'est publiée : le dépôt ne contient ni fichier de financement, ni canal de signalement de faille.

Deux risques ressortent en rouge — la licence et la dépendance à une seule personne — et un en orange, l'absence de canal de signalement de faille.

Les deux risques bas, la chaîne de vérification automatique et les 14 composants extérieurs, sont précisément ceux qui coûtent le plus cher à rattraper après une acquisition, et ils sont déjà tenus.

Les deux risques hauts, eux, ne se règlent pas par du code mais par contrat : une relicence signée et un engagement de l'auteur.

Pourquoi oui, pourquoi non ?

POINT CLÉ

Une technique qui tient et un gain chiffré, en face d'une demande nulle, d'un homme-clé unique et d'une licence fermée : le dossier n'est pas prêt, il est observable.

Pourquoi oui :

- Le gain est mesuré, pas promis : 13 à 34 % de jetons en moins sur les tâches difficiles, et 11 agents sur 11 qui adoptent l'outil sans qu'on le leur demande.
- Le marché sous-jacent grandit mécaniquement avec la dépense d'IA des équipes de développement, et le produit attaque directement la ligne de coût.
- La qualité d'exécution est vérifiable : contrôles automatiques sur trois systèmes, dépendances tenues, 14 paquets seulement.
- L'angle auto-hébergé répond à une objection de conformité que les indexeurs distants ne peuvent pas lever.

Pourquoi non :

- Aucune preuve de demande : 0 étoile, 0 copie, 1 demande ouverte sans commentaire, près de six mois après l'ouverture.
- Un seul contributeur, sur un compte personnel, sans financement ni relais désigné : le risque d'homme-clé est maximal.
- La licence AGPL-3.0 ferme la voie d'un produit vendu tant qu'elle n'est pas renégociée.
- Le public installe une version de mars quand le travail interne est de septembre : le produit démontré n'est pas le produit livrable.

Thèse en 3 lignes

1. **Ne pas investir aujourd'hui.** Le produit fonctionne et le gain est mesuré, mais il n'existe aucune preuve de demande extérieure : 0 étoile, 0 copie, 1 sollicitation sans réponse, presque six mois après l'ouverture.
2. **Trois conditions rendraient le dossier discutable**, dans cet ordre : une version publiée à jour, la 0.5.0 plutôt que la v0.1.6 de mars ; une relicence négociée avec l'auteur, seul détenteur des droits, pour sortir de l'AGPL-3.0 ; et dix utilisateurs extérieurs identifiés.
3. **Si l'une de ces conditions tombe, ce n'est pas une société qu'on achète mais une personne** : le vrai objet de la transaction est alors un recrutement ou une acquisition d'actif, valorisé comme tel, avec l'auteur engagé et une clause de non-départ.

Sources

1. https://github.com/jordanvalnet/code-dev-intel.ts/blob/main/docs/benchmarks/2026-06-07-agent-token-economy.md	2. https://github.com/jordanvalnet/code-dev-intel.ts/milestones	3. https://github.com/search?q=typescript+code+intelligence+mcp&type=repositories
--	--	--